
University of Buea
Faculty of Engineering and Technology
Department of Computer Engineering

CEF 440
Internet and Mobile Programming

Mobile App Development Process

Group 12 Project Report Task 1

Supervisor: **Dr. Valery Nkemeni**
Academic Year: 2025 / 2026, Second Semester

Contents

Executive Summary	3
1. Types of Mobile Applications	4
1.1 Native Applications	4
1.2 Progressive Web Apps	4
1.3 Hybrid Applications	5
1.4 Comparative Overview	5
2. Mobile Apps Programming Languages	6
2.1 Native Development Languages	6
2.2 Cross-Platform Languages	7
2.3 Hybrid Development Languages	7
2.4 Language Reference Table	8
3. Mobile App Development Frameworks	9
3.1 Native Frameworks	9
3.2 Cross-Platform Frameworks	10
3.3 Hybrid Frameworks	11
3.4 Framework Comparison	11
4. Mobile App Architectures and Design Patterns	12
4.1 MVC	12
4.2 MVP	13
4.3 MVVM	13
4.4 Clean Architecture	14
4.5 BLoC	14
4.6 Redux	15
5. Requirement Engineering	16
6. Mobile App Development Cost Estimation	18

Executive Summary

Scope of this report

This report addresses Task 1 of CEF 440 by examining the six core topics that precede any mobile software project: the categories of mobile applications, the programming languages available, the major development frameworks, the architectural patterns that govern code organisation, the requirement engineering process, and the methods used to estimate project cost.

Each section draws on material contributed by group members and expands it with sufficient technical depth to support the subsequent project phases. The report does not advocate for a single 'best' technology, the right choice in any of these areas depends on platform requirements, team expertise, and budget constraints, and these trade-offs are made explicit throughout.

1. Types of Mobile Applications

Mobile applications fall into three structural categories determined by how they are built, how they interact with the host operating system, and how they are distributed. The category decision shapes performance ceilings, development cost, and the range of device features accessible to the application.

1.1 Native Applications

Native applications are compiled specifically for a single operating system — Android or iOS — using languages and toolchains endorsed by the platform owner. The resulting binary runs directly on the device hardware without an intermediary layer, which is why native apps consistently deliver the highest performance and the most faithful implementation of platform design guidelines.

Examples: *Instagram, Spotify, Google Maps*

Strengths

- Full, immediate access to all device APIs — camera, GPS, biometric sensors, Bluetooth, NFC
- Peak runtime performance; animations and data processing operate at native speed
- Reliable offline behaviour with no dependency on a browser engine

Constraints

- Separate codebases for Android and iOS raise development and maintenance cost
- Requires specialists in platform-specific languages (Kotlin/Java and Swift/Objective-C)
- Store submission and review processes delay updates reaching users

1.2 Progressive Web Apps

Progressive Web Apps (PWAs) are web applications built with HTML, CSS, and JavaScript that expose a native-like experience through the browser. They are not installed from an app store; instead, users access them via a URL and may optionally add a shortcut to the home screen. Service workers — a background script layer — enable offline caching and push notifications, the two features that most distinguish PWAs from ordinary websites.

Examples: *Starbucks, Pinterest, Twitter Lite*

Strengths

- Updates are instant and server-side — no user action required and no store approval needed
 - Single codebase serves every device and screen size
 - Negligible installation overhead; no storage footprint on the device
-

Constraints

- Access to low-level hardware APIs (Bluetooth, NFC, background processing) is limited by browser support
- Offline capability is shallower than a comparable native application
- Discoverability via app stores is absent without additional distribution effort

1.3 Hybrid Applications

Hybrid applications are built using web technologies — HTML, CSS, JavaScript — and packaged inside a native shell that is distributed through official app stores. The shell provides a WebView component that renders the web-based UI, and a JavaScript bridge exposes selected device APIs to the web layer. This approach allows teams to write one codebase while still appearing in the App Store and Play Store.

Examples: *Twitter (early versions)*, *Uber Eats*

Strengths

- Significantly faster to build and ship than a full native implementation
- Distributed through official stores — users install and update them like any other app
- Accessible to web developers without native platform training

Constraints

- JavaScript bridge overhead reduces performance, especially for animations and real-time data
- Access to platform APIs depends on the availability of plugins, which vary in quality

1.4 Comparative Overview

Feature	Native	Progressive Web App	Hybrid
User Experience	Seamless, platform-native	Good, browser-dependent	Solid, some latency
Development Cost	High, two codebases	Low, single codebase	Medium
Distribution	App stores	Web browser / URL	App stores
Update Delivery	Store review required	Instant, server-side	Store review required
Offline Capability	Excellent	Limited	Good
Hardware Access	Full	Limited by browser	Partial via plugins
Performance	Highest	Medium	Medium

Table 1.1 — Structural comparison of the three mobile application categories.

2. Mobile App Programming Languages

The programming language selected for a mobile project determines the toolchain, the available libraries, the depth of platform integration, and to a large degree the team composition required. Mobile development organises around three approaches — native, cross-platform, and hybrid — each with its own linguistic conventions.

2.1 Languages for Native Development

Native development demands platform-specific languages that the operating system's runtime and compiler expect.

Kotlin

Kotlin is the primary language for Android development, officially endorsed by Google since 2017. It compiles to JVM bytecode and is fully interoperable with Java, meaning Kotlin code can call any existing Java library without modification. The language enforces null safety at compile time — eliminating an entire class of runtime crashes common in Java — and its concise syntax reduces boilerplate substantially. Android Studio provides first-class Kotlin tooling, and Jetpack Compose, the modern Android UI framework, is designed around Kotlin's language features.

Java

Java was the original language of Android development and remains pervasive across the ecosystem. Google's shift in preference toward Kotlin has not eliminated Java — the two languages coexist, and a large proportion of open-source Android libraries and enterprise codebases are still written in Java. Developers entering Android work should have a working knowledge of both.

Swift

Swift is Apple's primary language for iOS, macOS, watchOS, and tvOS development. Introduced in 2014 to replace Objective-C in most contexts, Swift emphasises safety through strong type inference, optional types, and automatic reference counting for memory management. Its syntax is expressive and comparatively readable, and Xcode provides integrated tooling including a live preview system for SwiftUI interfaces.

Objective-C

Objective-C was the dominant iOS language for over two decades and remains present in a large volume of legacy codebases. New projects almost universally use Swift, but any team maintaining or extending older applications must work with Objective-C. It uses a message-passing syntax inherited from Smalltalk, which differs markedly from most contemporary languages.

2.2 Cross-Platform Languages

Cross-platform languages allow a shared codebase to produce applications for both Android and iOS — and often web and desktop as well — from a single development effort.

Dart

Dart is the language of Flutter. It compiles ahead-of-time to native ARM code for mobile targets, which removes the JavaScript-to-native bridge overhead that limits some competing frameworks. Dart's syntax draws from Java and C#, making it approachable for developers with background in either. Its type system is sound — type errors are caught at compile time rather than at runtime.

JavaScript and TypeScript

JavaScript underpins React Native and Ionic, two of the most widely used cross-platform and hybrid frameworks. TypeScript is a statically typed superset of JavaScript that adds compile-time type checking, improved IDE tooling, and better support for large codebases. Most production React Native projects now default to TypeScript. The JavaScript ecosystem is exceptionally large, which means that libraries for almost any problem already exist.

C#

C# is Microsoft's object-oriented language and the basis for .NET MAUI, the successor to Xamarin. It is well-suited to teams already embedded in the Microsoft development ecosystem and to organisations building enterprise-grade applications where integration with Azure services, Active Directory, or Windows infrastructure is a requirement.

2.3 Hybrid Development Languages

Hybrid development relies on web technologies rather than compiled languages. HTML, CSS, and JavaScript are the standard trio. Python, through frameworks like Kivy and BeeWare, can produce mobile applications but is not widely used in professional mobile development; its more natural home is desktop tools and prototypes.

2.4 Language Reference Table

Language	Platform	Type	Difficulty	Performance	Ecosystem
Java	Android	Native	Medium	High	Mature
Kotlin	Android	Native	Easy–Medium	High	Growing rapidly
Swift	iOS / Apple	Native	Medium	High	Strong
Objective-C	iOS	Native (legacy)	Hard	High	Declining
Dart	Both	Cross-platform	Easy–Medium	High	Growing rapidly
JavaScript	Both	Cross / Hybrid	Easy	Medium	Very large

Language	Platform	Type	Difficulty	Performance	Ecosystem
TypeScript	Both	Cross-platform	Medium	Medium	Large
C#	Both	Cross-platform	Medium	Med-High	Enterprise
Python	Limited	Hybrid / Prototype	Easy	Low-Med	Niche mobile

Table 2.1 — Programming language characteristics across mobile development contexts.

3. Development Frameworks

A development framework is a structured collection of libraries, tools, and conventions that accelerates software construction by providing tested solutions to recurring problems. In mobile development, the framework choice determines the rendering pipeline, the available UI components, the build system, and the path to platform API access. Framework selection should follow — not precede — a clear definition of performance requirements, target platforms, and team skills.

3.1 Native Frameworks

Android Jetpack

Jetpack is Google's official library suite for modern Android development. It is not a single framework but a curated collection of components that each solve a specific problem:

- **Jetpack Compose** — a declarative UI toolkit that replaces XML layouts with Kotlin functions. State changes automatically trigger recomposition of the relevant UI nodes.
- **ViewModel** — manages UI-related state across configuration changes such as screen rotation, preventing unnecessary network calls and data loss.
- **Room** — an abstraction layer over SQLite that enforces schema correctness at compile time and supports coroutine-based asynchronous queries.
- **Navigation Component** — handles the back stack, deep links, and transitions between screens through a declarative navigation graph.
- **WorkManager** — schedules deferrable background tasks that are guaranteed to execute even if the app is closed or the device restarts.
- **Hilt** — a dependency injection library built on Dagger that reduces boilerplate while maintaining compile-time safety.

SwiftUI and UIKit

UIKit has been the UI framework for iOS since the first iPhone SDK in 2008. It uses a view-controller architecture where UI elements are arranged imperatively — developers write code that explicitly modifies the view hierarchy in response to events. A vast body of existing iOS code is written against UIKit.

SwiftUI, introduced in 2019, is Apple's declarative alternative. A SwiftUI view is a description of what the interface should look like for a given state; the framework handles all rendering updates. SwiftUI supports all Apple platforms — iPhone, iPad, Mac, Apple Watch, and Apple TV — from a single codebase, and integrates live preview directly inside Xcode. For new projects targeting iOS 16 and above, SwiftUI is the preferred choice.

3.2 Cross-Platform Frameworks

Flutter

Flutter, developed by Google, takes an unconventional approach to cross-platform rendering: rather than mapping UI components to native platform widgets, it renders everything through its own graphics engine (Skia on older systems, Impeller on newer ones). This means that a Flutter application looks identical on Android and iOS — there is no risk of subtle platform-specific rendering differences. The trade-off is that Flutter apps do not automatically adopt platform UI changes when the OS updates; the framework's own widget library must be updated instead.

Flutter's Hot Reload feature recompiles changed code and injects it into the running app in under a second, without losing application state. This feedback loop significantly compresses the UI iteration cycle.

React Native

React Native, maintained by Meta, maps JavaScript component declarations to the native UI widgets of each target platform. An Android React Native button renders as a native Android button; an iOS one renders as a native iOS button. The New Architecture introduced JSI (JavaScript Interface), Fabric (a new rendering system), and TurboModules (lazy-loaded native modules), collectively reducing the overhead of the JavaScript bridge that constrained earlier versions. React Native benefits from the enormous JavaScript ecosystem and from the large pool of developers already familiar with React.

.NET MAUI

.NET Multi-platform App UI is Microsoft's evolution of Xamarin, targeting Android, iOS, macOS, and Windows from a single C# codebase running on the .NET 6+ runtime. MAUI maps to native controls on each platform, which preserves platform-native appearance while sharing business logic. It fits naturally into Microsoft Azure ecosystems and is particularly used in enterprise environments.

Kotlin Multiplatform

Kotlin Multiplatform (KMP) takes a different approach from Flutter and React Native: it shares only the business logic layer across platforms, leaving each platform's UI entirely native. Android uses Jetpack Compose; iOS uses SwiftUI or UIKit. The shared Kotlin module contains networking, data parsing, state management, and any other logic not specific to a UI. This gives teams maximum native UI fidelity while eliminating the duplication of non-UI code.

3.3 Hybrid Frameworks

Ionic and Capacitor

Ionic provides a library of mobile-optimised web components styled to approximate native UI conventions. Capacitor — the modern successor to Apache Cordova — is the native runtime that wraps the Ionic web application and exposes device APIs to JavaScript through a clean plugin architecture. Capacitor also supports deployment as a PWA from the same codebase, making it viable for teams targeting both web and mobile audiences simultaneously.

Apache Cordova

Cordova is the original hybrid bridge and the predecessor to Capacitor. It remains maintained and present in many legacy projects, but new projects should prefer Capacitor for its improved performance, simpler plugin API, and better support for modern web standards.

3.4 Framework Comparison

Framework	Type	Language	Performance	Code Sharing
Android Jetpack	Native	Kotlin	Highest	Android only
SwiftUI / UIKit	Native	Swift	Highest	Apple platforms
Flutter	Cross-platform	Dart	High	~95 %
React Native	Cross-platform	JavaScript	Med–High	~85 %
.NET MAUI	Cross-platform	C#	Med–High	~80 %
Kotlin Multiplatform	Cross-platform	Kotlin	High	Logic layer only
Ionic + Capacitor	Hybrid	HTML/CSS/JS	Medium	~90 %+

Table 3.1 — Framework comparison across type, language, and expected sharing.

4. Architectures and Design Patterns

An architectural pattern imposes structure on a codebase: it defines what categories of code exist, where each lives, and how they may communicate. Without deliberate architecture, applications accumulate tightly coupled logic that becomes progressively harder to test, extend, or hand to another developer. The six patterns described here represent the dominant approaches in contemporary mobile development — each exists to solve a specific set of structural problems.

4.1 MVC (Model, View, Controller)

MVC is the oldest pattern in this list and the one most developers encounter first. The application is divided into three components:

- **Model** — owns all data and data-access logic. In a banking application, the Model holds account balances and transaction records.
- **View** — renders everything visible to the user: layouts, text fields, buttons.
- **Controller** — mediates between the other two. It receives user events from the View, queries or updates the Model accordingly, and instructs the View on what to display.

In practice, the Controller accretes responsibility as the application grows. Navigation logic, input validation, analytics calls, and UI update logic all migrate there because no other component is an obvious fit. Developers refer to the result as the 'Massive View Controller' problem. MVC remains a reasonable starting architecture for small, bounded screens, but it struggles at scale.

4.2 MVP (Model, View, Presenter)

MVP addresses Controller bloat by making the View entirely passive. The View has one job: report user events (a button tap, a text entry) to its Presenter. Every decision about what to display, and in what sequence, belongs to the Presenter.

Because the Presenter holds no reference to any Android Activity or iOS ViewController, only to an abstract View interface, it can be instantiated and tested in isolation. All business behaviour can be verified by unit tests that never touch the device. This testability improvement made MVP the dominant Android pattern for several years before MVVM superseded it.

4.3 MVVM (Model, View, ViewModel)

MVVM is the current default pattern in Flutter, Jetpack Compose, and SwiftUI. Its distinguishing mechanism is reactive data binding: the ViewModel exposes observable state, and the View subscribes to it. When the ViewModel updates its state, the UI re-renders automatically — no explicit method call from the ViewModel to the View is required.

The ViewModel holds no reference to the View. It cannot import or instantiate any UI class. This constraint enforces a clean separation and makes ViewModel logic straightforward to test independently of the rendering layer.

The automatic UI updates that data binding provides reduce the likelihood of bugs caused by the View and the data falling out of sync — a common problem in MVC and early MVP implementations. MVVM is the most widely recommended architecture for new mobile projects across all major platforms.

4.4 Clean Architecture

Clean Architecture, codified by Robert C. Martin, extends the separation-of-concerns principle beyond the UI layer. The application is structured as three concentric layers governed by one rule: dependencies point inward only. Outer layers may depend on inner layers; inner layers are forbidden from knowing anything about outer ones.

- **Presentation Layer:** ViewModels and UI components. Changes frequently as design requirements evolve.
- **Domain Layer:** business rules and use cases. Defines what the application does, independently of any specific technology. This layer has no imports from Android, iOS, or any database library.
- **Data Layer:** repository implementations, API clients, and local database access. Provides data to the Domain layer through interfaces defined by the Domain layer itself.

The practical consequence: the Domain layer's rule that a transfer cannot exceed an account balance is unchanged whether the application is accessed via a mobile app, a web interface, or a command-line tool. Clean Architecture is the standard structural approach for large, multi-developer projects intended to survive years of platform changes.

4.5 BLoC (Business Logic Component)

BLoC is a pattern developed by the Google Flutter team and formalised in the `flutter_bloc` library. It enforces strict unidirectional data flow: the UI dispatches Events into the BLoC; the BLoC processes them and emits States; the UI re-renders in response to those States.

This structure makes the history of an application's state fully traceable. Every state transition is the result of a recorded event, which makes debugging and testing straightforward. BLoC is particularly effective in applications with complex shared state — a shopping cart visible from multiple screens, or a user authentication status that affects navigation throughout the app.

4.6 Redux

Redux originated in the React ecosystem and is widely adopted in React Native projects. The entire application state lives in a single immutable object — the Store. Nothing modifies the Store directly.

When something happens, the application dispatches an Action — a plain data object describing the event, such as `USER_LOGGED_IN` or `ITEM_REMOVED_FROM_CART`. A Reducer function receives the current state and the Action, and returns a new state object. The Store updates to this new state, and all subscribed UI components re-render.

Because the Store is immutable and every change is produced by a pure Reducer function, any state in Redux's history can be reproduced exactly. This property makes time-travel debugging — stepping backwards through state transitions — practical.

Redux introduces meaningful boilerplate for small applications, but in large React Native projects where dozens of screens share overlapping data, its predictability and tooling ecosystem (Redux DevTools, Redux Toolkit) justify the overhead.

Pattern	Key Principle	Best Suited For	Testability
MVC	Three roles: Model, View, Controller	Small apps, rapid prototypes	Moderate
MVP	Passive View; Presenter decides all	Android legacy, high unit-test coverage	High
MVVM	Reactive binding; ViewModel has no View	Modern Android, Flutter, SwiftUI	Very High
Clean Arch.	Dependencies point inward only	Large, long-lived enterprise apps	Very High
BLoC	Unidirectional: Events in, States out	Complex shared state in Flutter	Very High
Redux	Single global immutable Store	Large React Native apps	High

Table 4.1 — Architectural patterns summarised by principle, context, and testability.

5. Requirement Engineering

Requirement engineering is the structured process of discovering, analysing, documenting, and validating what a software system must do before any implementation begins. On mobile projects, inadequate requirements are among the most common causes of cost overrun, rework, and user rejection. A requirements phase that surfaces ambiguities early is far cheaper than discovering them after a working feature has been built and shipped.

5.1 Stakeholder Identification

Before any elicitation begins, the team must identify everyone whose goals, constraints, or working practices the application will affect. This typically includes end users (the primary audience for features and usability), the business owner or product sponsor (who defines priorities and success metrics), the development team (who identifies technical constraints), quality assurance engineers, and — for applications subject to regulation — compliance or legal representatives.

Each stakeholder class has different information to contribute and different terminology. A requirements analyst who speaks only to the business owner will miss the usability concerns surfaced by actual users; one who speaks only to engineers will miss the business rules that should govern the system's behaviour.

5.2 Elicitation Techniques

Elicitation is not passive collection — it requires deliberate effort to surface requirements that stakeholders have not articulated, sometimes because they consider them obvious, sometimes because they have not yet recognised them as requirements. Several techniques are typically combined:

1. **Structured interviews** — one-to-one sessions with key stakeholders following a prepared question set. Effective for understanding decision-making processes and business rules.
 2. **Surveys and questionnaires** — distributed to larger user populations where individual interviews would be impractical. Useful for identifying frequency-of-use patterns and priority rankings.
 3. **Observation and contextual inquiry** — watching users perform tasks in their actual working environment. Frequently reveals unstated assumptions and workarounds that would not surface in an interview.
 4. **Workshops and brainstorming** — structured group sessions that generate requirements collaboratively and surface conflicting stakeholder priorities early.
 5. **Reverse engineering** — analysing existing or competing applications to derive an expected baseline of functionality the new system should meet or exceed.
 6. **Prototyping** — building low-fidelity mockups or clickable wireframes that stakeholders can react to. Prototypes convert abstract requirements into concrete discussions.
-

5.3 Requirement Analysis

Once a body of raw requirements has been collected, they must be analysed for quality before they can be used. A requirement that is ambiguous, contradictory, or technically infeasible is not a requirement — it is a liability. Analysis involves:

- **Completeness** — verifying that all necessary behaviours are specified and that no functional area has been omitted.
- **Consistency** — identifying and resolving requirements that contradict one another.
- **Feasibility** — confirming that each requirement can be implemented within the constraints of the project's technology stack, timeline, and budget.
- **Classification** — separating functional requirements (what the system does) from non-functional requirements (performance thresholds, security standards, accessibility targets, and similar quality attributes).
- **Prioritisation** — ranking requirements by business value and implementation complexity so that the highest-value, lowest-risk work is delivered first.

5.4 Software Requirements Specification

The Software Requirements Specification (SRS) is the formal output of the requirements phase. It records all agreed requirements in a structured, unambiguous form and serves as the shared contract between the development team and the client. A well-written SRS supports every subsequent project activity: system design references it to determine architecture; testers derive test cases from it; project managers use it to scope work and estimate cost.

A typical SRS contains: an introduction and system overview; functional requirements organised by feature area; non-functional requirements covering performance, security, and usability; use case diagrams and user stories; and a glossary that establishes consistent terminology across all stakeholders.

5.5 Validation

Documented requirements must be validated with stakeholders to confirm that they accurately represent actual needs. Validation methods include formal review walkthroughs, acceptance testing against prototype builds, and sign-off sessions where stakeholders confirm that the SRS reflects their intent. Requirements that pass validation without this confirmation step frequently turn out to be someone's assumption rather than a genuine stakeholder need.

6. Cost Estimation

Estimating the cost of a mobile application is not a single calculation — it is a structured reasoning process that requires a clear understanding of scope before any number can be credible. A development team that skips this process produces estimates that are invariably too low, because they exclude entire categories of work that are not visible until the project is already under way.

6.1 Define the Scope

The first and most important input to any estimate is a well-defined scope. Scope encompasses the target platform or platforms (iOS, Android, or both), the development approach (native, cross-platform, or hybrid — with native consistently more expensive), the feature set of the initial release, the complexity of the UI/UX design, and any backend infrastructure the app must communicate with or depend upon.

A vague scope produces a vague estimate. Every assumption embedded in the scope definition should be made explicit and agreed upon before work begins.

6.2 Decompose Features by Complexity

Each feature is broken down into implementation tasks and assigned an estimated effort. Three bands are standard:

Complexity Band	Estimated Hours	Representative Features
Simple	20 – 40 hours	Static information screens, basic navigation, login and logout
Medium	40 – 80 hours	User profiles, push notifications, map integration, search
Complex	80 + hours	Real-time messaging, payment processing, offline sync, ML features

Table 6.1 — Feature complexity bands and indicative hour ranges.

6.3 Apply Hourly Rates

Development rates vary with two primary factors: developer seniority and geographic location. Both must be considered when assembling a realistic budget.

Seniority	Hourly Rate (USD)
Junior developer	\$25 – \$50
Mid-level developer	\$50 – \$100
Senior developer	\$100 – \$150+

Table 6.2 — Indicative hourly rates by developer seniority.

Region	Hourly Rate Range (USD)
Asia	\$15 – \$40
Latin America	\$40 – \$75
Eastern Europe	\$40 – \$85
Western Europe / North America	\$80 – \$150+

Table 6.3 — Indicative hourly rates by geographic region.

6.4 Calculate the Base Cost

With estimated hours and an applicable rate, the base development cost is:

$$\text{Total Development Cost} = \text{Estimated Hours} \times \text{Hourly Rate}$$

For example: an application estimated at 350 hours of development, staffed with mid-level developers at \$70/hour, produces a base cost of \$24,500.

6.5 Add Overhead and Contingency

The base figure covers only raw development time. A realistic project budget must also account for:

Cost Component	Typical Addition	Notes
Project management	+10 – 15 %	Coordination, sprint planning, client communication
Testing and QA	+10 – 15 %	Manual testing, automated test suites, regression cycles
UI/UX design	+10 – 20 %	Wireframes, interactive prototypes, visual design
App store fees	Fixed	Google Play: USD 25 one-time; Apple: USD 99 / year
Contingency buffer	+10 – 20 %	Scope changes, rework, integration surprises

Table 6.4 — Overhead categories and their typical contribution to total project cost.

Applying a combined overhead and contingency factor of 40–50 % to the base development cost produces an estimate that accounts for the full scope of project work:

$$\begin{array}{ccccccc} \text{Realistic Total Cost} & = & \text{Base Cost} & \times & & & \\ 1.40 & \text{to} & 1.50 & & & & \end{array}$$

Cost estimates should be revisited at the end of each project phase as scope and complexity become better understood. An estimate produced before requirement engineering is complete is a rough order of magnitude only; it should not be treated as a commitment.
